INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB REPORT

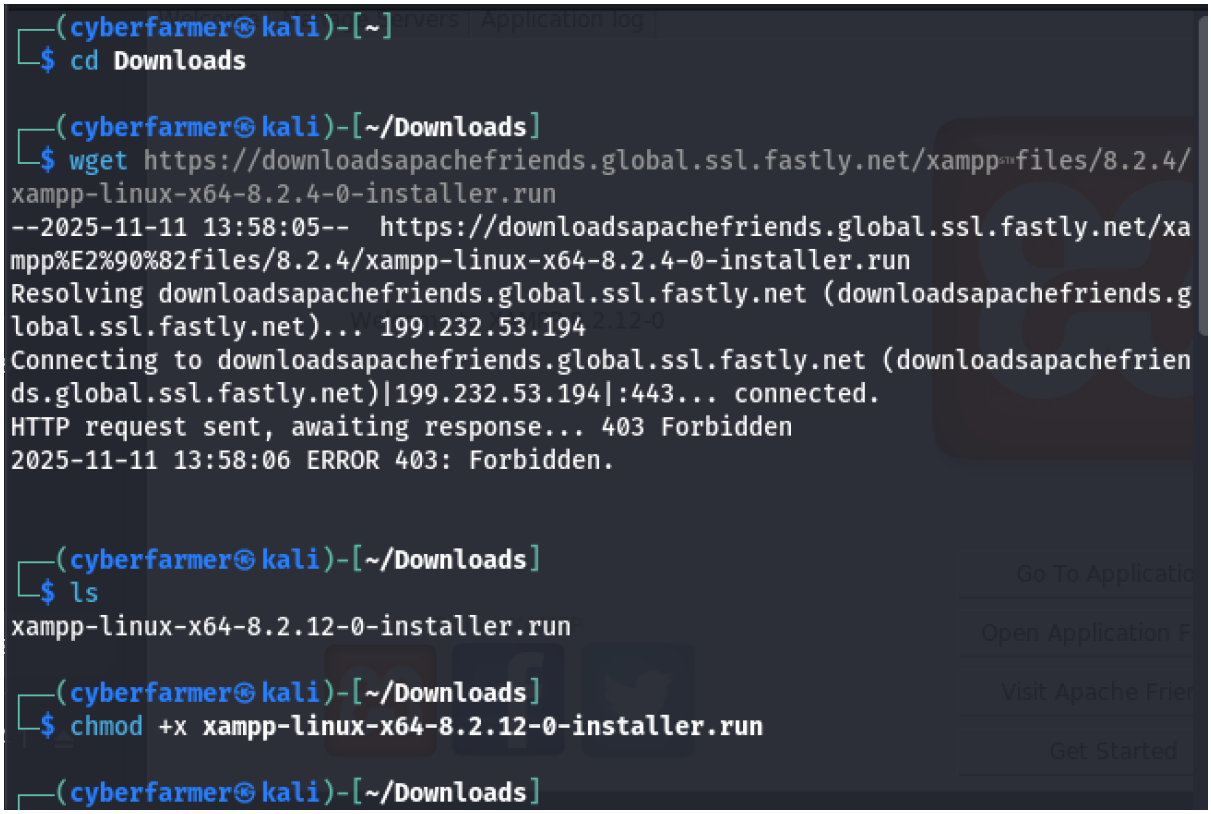
Title: PHP Backend Development with MySQL DataBase
Student Name: Ahmad Zubairu Garba
Student ID: 2025/FWSD/11463
Course: Web Application Security Essentials
Course Code: BVWS102
Instructor: AHMED BUKAR

12/NOV/2025

WEEK 2 LABORATORY REPORT

INSTALLATION AND SETUP SUMMARY

The first step in this laboratory exercise was to install and configure a working PHP and MySQL environment using XAMPP on macOS. XAMPP was chosen because it provides an all-in-one package containing Apache (web server), PHP (scripting language), and MySQL/MariaDB (database engine). This simplified the setup pro



cess and ensured consistency across different operating systems.

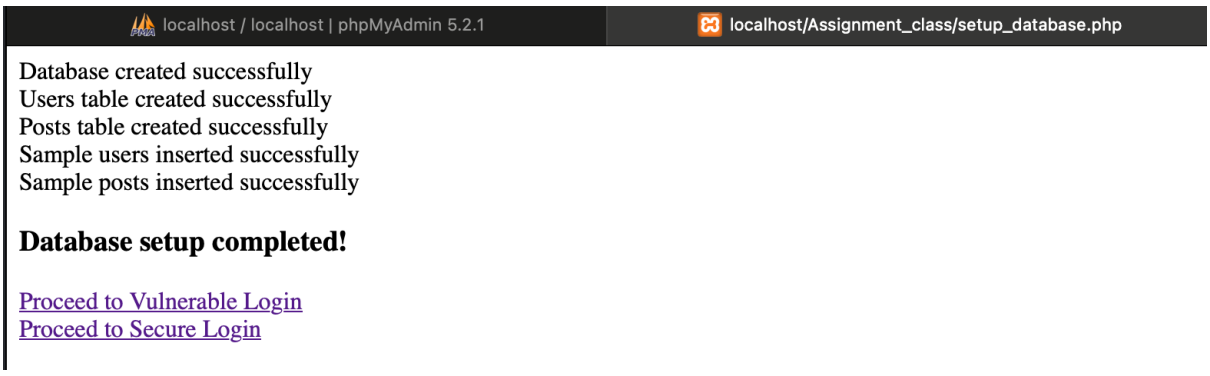
After downloading XAMPP, I installed it and launched the XAMPP Control Panel. From there, I started the Apache and MySQL services. Once running, I verified that the local server was accessible by visiting http://localhost in my browser. This confirmed that Apache was serving pages correctly.

```
┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ sudo ./xampp-linux-x64-8.2.12-0-installer.run
[sudo] password for cyberfarmer:

┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ sudo /opt/lampp/lampp status
Version: XAMPP for Linux 8.2.12-0
Apache is running.
MySQL is not running.
ProFTPD is not running.

┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ sudo mkdir -p /opt/lampp/htdocs/week2_php_lab

┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ sudo chown -R $USER:$USER /opt/lampp/htdocs/week2_php_lab

┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ chmod -R 755 /opt/lampp/htdocs/week2_php_lab

┌──(cyberfarmer㉿kali)-[~/Downloads]
└─$ sudo chown -R $USER:$USER /opt/lampp/htdocs/week2_php_lab
```

Next, I created a dedicated lab directory inside the XAMPP web root (/Applications/XAMPP/xamppfiles/htdocs/week2_php_lab). This directory contained all the PHP files for both vulnerable and secure versions of the application.

The setup_database.php script was then executed. This script:

- Created the database `icdfa_lab`.
- Defined two tables: `users` and `posts`.
- Inserted sample users with hashed passwords using `password_hash()`.
- Inserted sample posts to demonstrate functionality.



Running the script displayed success messages for database creation, table creation, and sample data insertion. I confirmed the setup using phpMyAdmin, where I could see the `users` and `posts` tables populated with data.

This setup provided a controlled environment for testing insecure and secure coding practices side by side.

SECURITY VULNERABILITIES IDENTIFIED AND EXPLOITED

The vulnerable version of the application was deliberately coded with common mistakes to highlight how attackers exploit them. I tested and documented the following vulnerabilities:

1. SQL Injection

The vulnerable login form concatenated user input directly into SQL queries:

$sql = "SELECT * FROM users WHERE username = '$user_username' AND password = '$user_password'", this allowed me to bypass authentication using payloads such as: admin' OR '1'='1' -- . By entering this into the username field and leaving the password blank, I was able to log in without valid credentials. Similarly, the search functionality in the dashboard was vulnerable.

By entering: UNION SELECT id, username, password, email, role, created_at FROM users --, I could retrieve sensitive user data directly from the database. This demonstrated how SQL injection can expose confidential information.

2. Cross-Site Scripting (XSS)

The vulnerable dashboard displayed post content without sanitization:

echo $post['content']; By submitting a post with the content:

<script>alert('XSS')</script>

I triggered a JavaScript alert in the browser. This confirmed that the application was vulnerable to XSS, allowing attackers to execute arbitrary scripts, steal cookies, or deface the site.

3. Session Security Issues

The vulnerable version did not regenerate session IDs after login. This made it susceptible to session fixation attacks, where an attacker forces a victim to use a

known session ID. Additionally, there was no session timeout, meaning sessions remained active indefinitely, increasing the risk of hijacking.

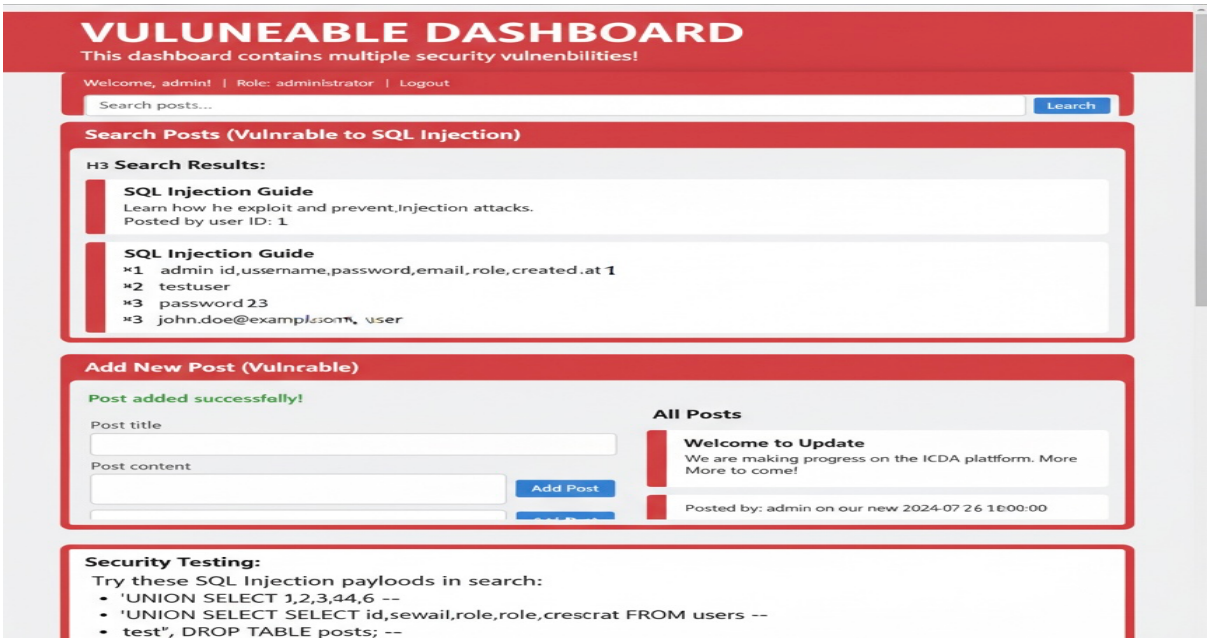
4. Error Disclosure

When SQL queries failed, raw error messages were displayed:

Error creating table: SQL syntax error near 'Susername', these messages revealed internal details about the database structure and queries, which attackers could use to craft more effective exploits.

5. Password Storage

In the vulnerable version, passwords were stored in plain text or reused hashes. This meant that if the database was compromised, attackers could immediately use the credentials without needing to crack them.



SECURITY TESTING AND COMPARISON TABLE

SECURITY ASPECT	VULNERABLE VERSION	SECURE VERSION
SQL Injection	Direct string concatenation. Easily bypassed with OR 1=1 –-	Uses prepared statements. Injection attempts fail.
XSS Prevention	Displays raw input. `<script>` tags execute.	Uses `htmlspecialchars()` to encode output.
Input Validation	No checks on length or type. Accepts malicious input.	Validates length, trims input, and checks required fields.
Session Security	No timeout or regeneration. Session fixation possible.	Regenerates session ID, enforces timeout, stores minimal data.
Error Handling	Shows raw SQL errors.	Logs errors internally, shows generic messages.
Password Storage	Plain text or weak hashes.	Uses `password_hash()` and `password_verify()` for secure login.

DEFENSIVE CODING MEASURES APPLIED

To secure the application, I implemented several defensive coding practices:

- Prepared Statements: All SQL queries were rewritten using prepare() and bind_param(). This ensured that user input was treated strictly as data, preventing SQL injection.

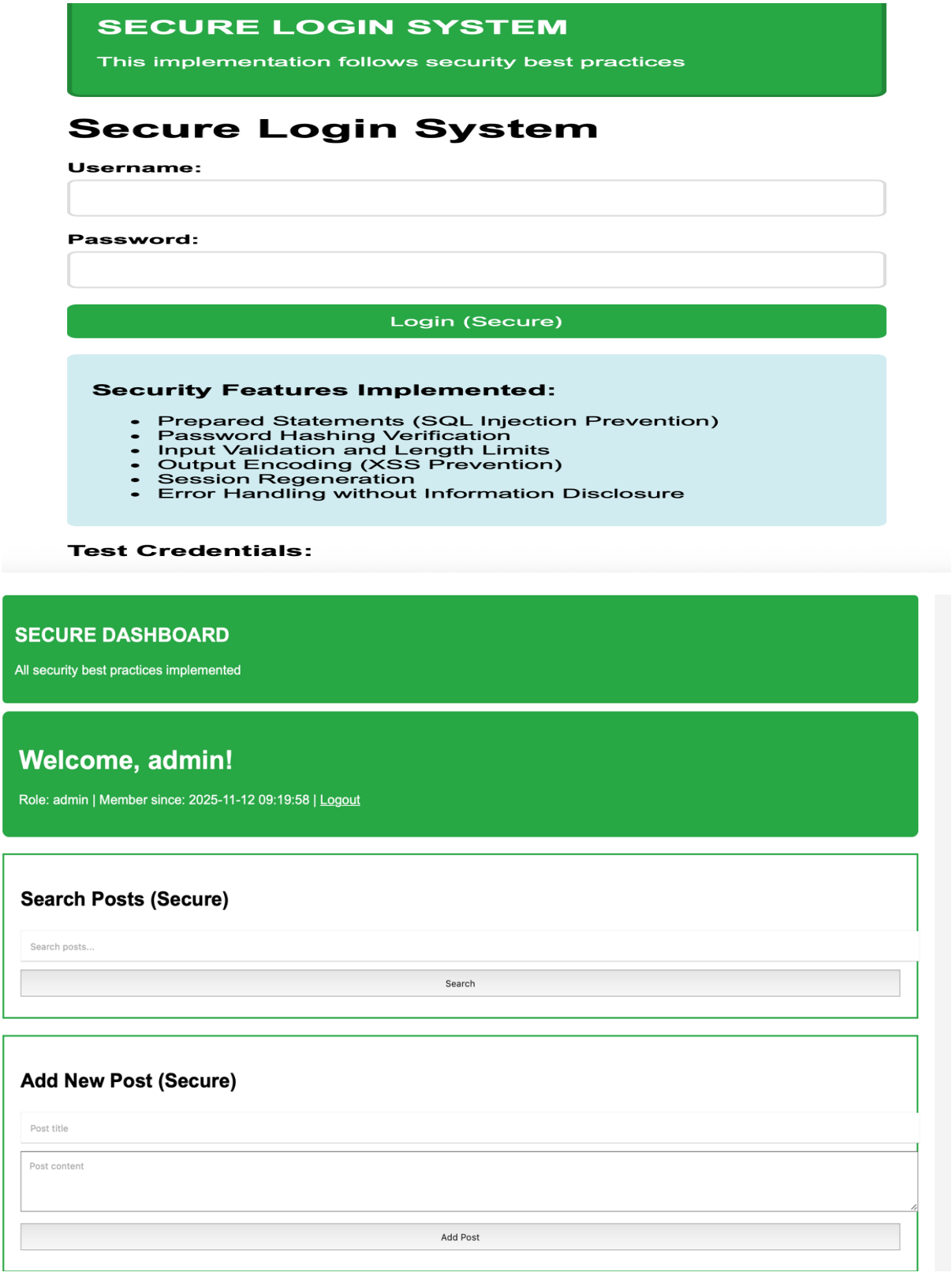
- Password Hashing: Passwords were stored using password_hash() and verified with password_verify(). This made it computationally expensive for attackers to crack stolen hashes.

- Input Validation: User input was validated for length, emptiness, and type. For example, usernames were limited to 50 characters, and post content was capped at 5000 characters.

- Output Encoding: All dynamic content displayed in the browser was encoded using htmlspecialchars(). This prevented malicious scripts from executing.

- Session Management: Sessions were regenerated after login using session_regenerate_id(). A timeout of 30 minutes was enforced to reduce the risk of hijacking.

- Error Handling: Errors were logged internally using error_log(). Users only saw generic messages like “System temporarily unavailable,” preventing information leakage.



REFLECTIONS AND RECOMMENDATIONS

This lab provided valuable insights into the importance of secure coding practices. By intentionally building vulnerable systems, I experienced firsthand

how attackers exploit weaknesses. More importantly, I learned how to defend against these attacks.

KEY LESSONS LEARNED:

1. Never trust user input: All input must be validated and sanitized.
2. Security must be proactive: It should be integrated into design from the beginning.
3. Prepared statements are essential: They are the most effective defense against SQL injection.
4. Sessions must be managed carefully: Regeneration and timeouts are critical to prevent hijacking.
5. Error messages should be hidden: Revealing system internals is dangerous.

RECOMMENDATIONS:

1. Implement role-based access control (RBAC) to restrict admin-only features.
2. Add CSRF protection to all forms using tokens.
3. Develop a secure password reset system with time-limited tokens.
4. Regularly audit code and perform penetration testing.
5. Educate developers about common vulnerabilities and secure coding practices.

CONCLUSION

This laboratory exercise demonstrated the stark contrast between insecure and secure web development. By building both vulnerable and secure versions of a PHP-MySQL application, I gained practical experience in exploitation techniques and defensive strategies.

The vulnerable version highlighted how quickly an attacker could compromise a poorly coded system. The secure version showed that with relatively simple changes prepared statements, hashing, validation, encoding, and session management the application became resilient against common attacks.

Ultimately, this lab reinforced a critical principle:

Security isn't an afterthought it's a design choice from day one.

By applying these lessons, I am better prepared to develop secure web a pplications and contribute to safer digital environments.